



Build Your Own Battle Arena!

How this 3D game works — and how **YOU** can make it better

A friendly guide for curious kids 🎮

Made with code. No experience needed.

What is this game? 🎮

Defend your **3 towers**. Drop **cards** — knights, dragons, wizards — onto a battlefield split by a **river** and two **bridges**.

- Spend **elixir** (purple energy) to play cards
- Troops march and fight on their own
- Match lasts **3 minutes** → smash towers to win **crowns** 👑
- Destroy the enemy **King Tower** = instant win!

All **art and sounds are original**, made from simple shapes and computer sounds.

The 3D arena you build

This whole battlefield is drawn from **simple 3D shapes** — boxes, balls, cones:

- Two lanes, a river, two bridges
- 3 towers per side
- Troops walk, fight, and shoot
- Even the snow, trees, and moon are code!

➡ *Everything you see is built by the program, piece by piece.*



The Big Secret: Brain 🧠 + Body 💪

The **most important idea** in the whole game:

The Brain (rules)	The Body (what you see/hear)
Knight's health	What the knight looks like
Who attacks whom	The sword swing
Who wins	The victory music

Like **chess**: the rules are the same whether the pieces are wood, glass, or on a phone. We keep the **rules** and the **pictures** separate — it makes the game easy to fix and grow. 🌱

What is it made with?

Tool	Kid-friendly meaning
TypeScript	The language — like English for computers
Three.js	The "art teacher" that draws 3D shapes
Web Audio	A tiny music machine in your browser
Vite	The "play button" that starts the game
Vitest	The "homework checker" for the code

It all runs **inside a web browser** — no app store needed!

The map of the project

```
src/
├── game/
│   ├── cards.ts    every card's stats
│   ├── sim.ts      the battle "heartbeat"
│   └── bot.ts       the computer opponent
├── render3d/
├── audio/
├── net/
└── main.ts
```

 THE BRAIN: rules & math (no pictures!)

 THE BODY (eyes): draws the 3D world

 THE BODY (ears): makes sounds

 play a friend over Wi-Fi

 glue that connects it all

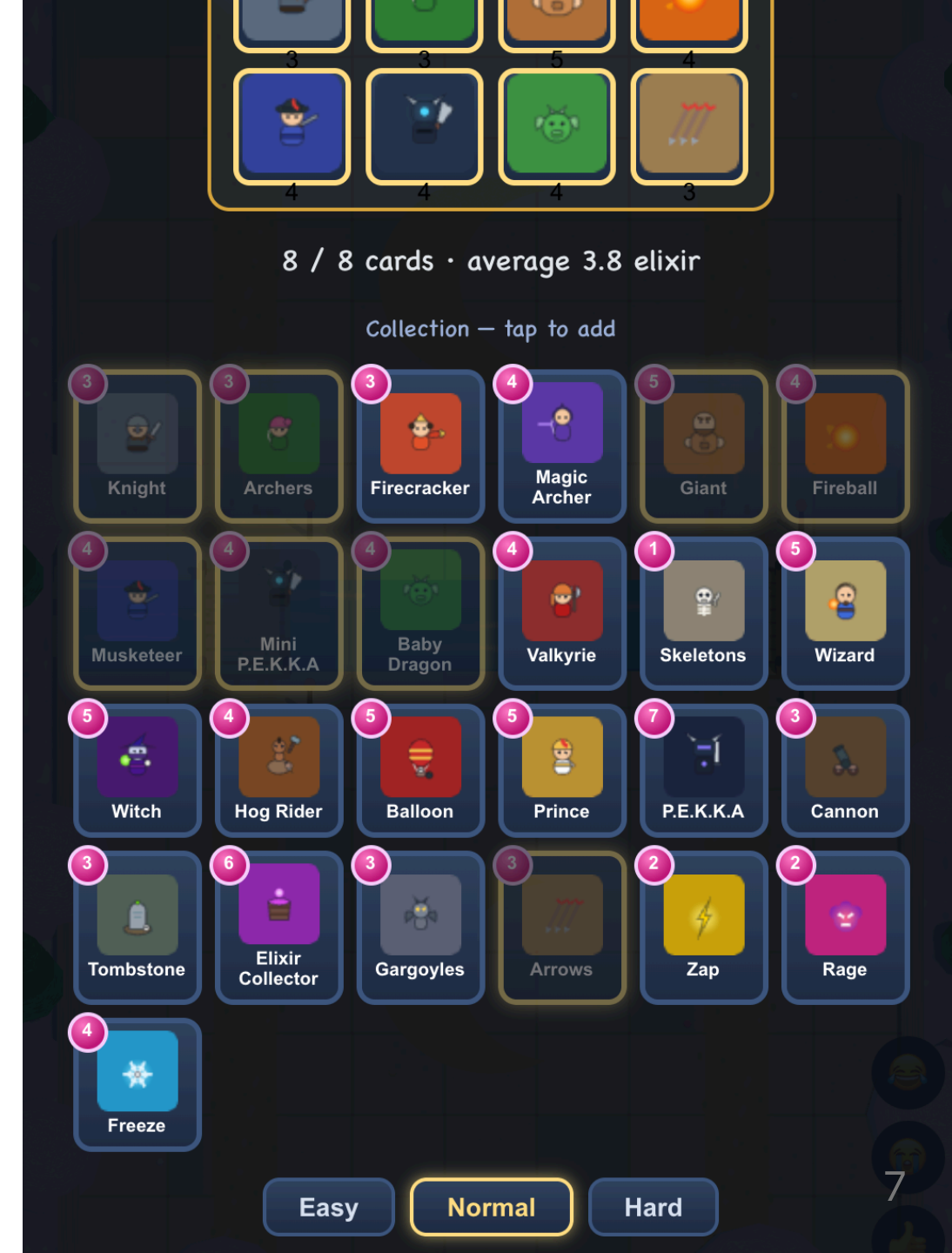
The **game/** folder never draws anything — that's the secret! ✨

These are the cards

You build a deck of **8 cards** from the whole collection.

Two of these — the  **Firecracker** and the  **Magic Archer** — were added recently, step by step, with code.

You can invent your own cards too! 



A card is just a list of facts

Open `src/game/cards.ts` — every card is simple facts:

```
knight: {  
  name: "Knight",  
  cost: 3,           // costs 3 elixir  
  unit: unit({  
    maxHp: 1400,      // health  
    damage: 160,      // how hard it hits  
    hitSpeed: 1.2,    // seconds between hits  
    attackRange: 0.8, // short = melee (must be close)  
    speed: "medium",  
  }),  
}
```

Change `maxHp: 1400` → `maxHp: 5000` and the Knight becomes a super-tank! 💪

The heartbeat of the battle

A game is like a **flipbook** — many pictures per second. Each page is a **tick**, run ~30 times a second in `sim.ts`:

```
export function tick(state, dt) {  
  // 1. Give both players elixir  
  // 2. Every unit: find a target, walk, attack  
  // 3. Move the flying arrows & fireballs  
  // 4. Remove anything that died  
  // 5. Check: did someone win?  
}
```

`dt` = time since the last tick. A slow computer takes **fewer, bigger** steps; a fast one takes **many tiny** steps — everyone covers the **same ground**. 💬 Like a **flipbook**: flip slow or fast, it's the same story. ⌚

How does a troop pick a fight?

Every tick, each troop asks:

1. **"Who's my target?"** (usually the nearest enemy)
2. **"Am I close enough?"** → **Yes:** attack  **No:** step closer 

We added **target persistence**: once a troop is *already* fighting, it **keeps** fighting — it won't get distracted by a new enemy.   

```
if (current && gap(e, current) <= e.attackRange) {  
    return current; // stay on it – don't get distracted!  
}
```

Two cards we built ✨

Firecracker

Shoots a **splashing** firework and **recoils** — hops backward after firing!

Magic Archer

Fires one arrow that **pierces** — hits every enemy in a straight line.

```
firecracker: {  
  name: "Firecracker", cost: 3,  
  unit: unit({  
    maxHp: 140, damage: 130, attackRange: 6,  
    speed: "fast", targetsAir: true,  
    splashRadius: 1.4, // sparks spread  
    recoil: 2,         // hops back after firing   
  }),  
}
```

^{1 2}_{3 4} Math: how far away is it?

Every tick, troops ask *"how far is my enemy?"* The answer is the **Pythagoras theorem** you learn at school! 

```
function distance(a, b) {  
    return Math.hypot(a.x - b.x, a.y - b.y); //  $\sqrt{dx^2 + dy^2}$   
}
```

The arena is a grid of **(x, y)** points. The straight-line distance is the **hypotenuse** of a right triangle — **dx** across, **dy** up. 

Real math, used 1000s of times every second!



Physics: how a troop moves

To move, we point an **arrow** (a *vector*) from the troop to its goal, shrink it to length **1** (*normalize*), then step along it:

```
const d = distance(e, goal);  
e.x += (goal.x - e.x) / d * speed * dt; // step a little each tick  
e.y += (goal.y - e.y) / d * speed * dt;
```

position = position + direction × speed × time 🕒

💬 Think of a **compass + footsteps**: the *direction* (the shrunk arrow) points the way, and *speed × time* says how far to step this tick. 🧭

Physics: bumping, kicking & walls

Real physics ideas live in the game:

- **Soft collisions** 🤝 — two troops can't stand in the same spot, so they gently shove apart. A big Giant pushes a tiny Skeleton more than the reverse.
- **Recoil** 💣 — the Firecracker *kicks herself backward* after firing. That's **Newton's 3rd law**: every push has an equal push back!
- **Invisible walls** 🚧 — we once found troops recoiling **off the edge** of the map! We added walls so a body can never leave the arena. 🧯

Finding and fixing that was real detective work — debugging! 🕵️

Programming: loops & recipes

Two ideas power everything:

- A **loop** does the same job for every item:

"**For every** troop: find a target, move, attack."

- A **function** is a **recipe** you name once and reuse forever —

`distance(...)`, `tick(...)`, `moveToward(...)`.

```
for (const troop of allTroops) {  
  actEntity(troop, dt); // run the same recipe on each one  
}
```

Write the recipe **once**, the computer repeats it **millions** of times. 

Architecture: the brain shouts, the body listens

How do the rules talk to the pictures **without** mixing them? With **events** — little notes the Brain shouts out loud: 📢

```
state.events.push({ type: "crown", winner: "player" }); // 🧠 "a tower fell!"
```

The **Body** is listening and reacts:

- 🧠 Ears → play the *crown* sound 🔔
- 👁 Eyes → shake the camera & show 🏰

The Brain never draws or makes sound; the Body never knows the rules — they only **share notes**. That's clean **architecture**. 🧩

💬 Like a **referee**: the whistle just shouts "*goal!*" — the players and the crowd decide how to celebrate. 🎉

No dice, no clock = the same game every time

The Brain **never** rolls real dice or peeks at the real clock. When it needs "random," it uses a **seed** — a start number that always makes the *same* sequence:

```
const rng = mulberry32(42); // same seed 42 → same "random" every time
```

This is called being **deterministic** — like a **recipe**: same ingredients and steps → the **same cake**, every single time. 🍰 Same moves → **same battle**. That makes bugs easy to **reproduce**... and powers multiplayer 🙌

Multiplayer: how a "room" works

Two kids, two phones, **one battle**. How do the screens stay matched?

- Each player **creates or joins a room** with a secret animal code 🦊
- The room is a tiny **mail carrier** — it only passes **moves** ("played Knight at the left bridge"), *never* the whole game
- Both phones run the **identical deterministic Brain** in **lockstep** 🔒

💬 Like two kids doing the **same dance** while one **caller** shouts the moves: the room is just the caller, passing along moves. Because the Brain has no dice and no clock, both phones make the **same game** — not a video! ✉️ ✨



AI: how the computer opponent thinks

The bot in `bot.ts` follows a **priority ladder** every second — the first that fits, it does:

1. 🎯 **Spell** a tight clump of my troops (only if it's worth the elixir)
2. 🛡️ **Defend** — answer an attacker (kite a tank with a *building*!)
3. 🏠 **Save up** — drop an Elixir Collector when safe and full
4. ⚔️ **Attack** — lead with a tank, then send **support** to its lane

We recently made it **smarter**: it now picks *one* lane and presses harder in double elixir, instead of dribbling troops everywhere. 🧠

🌀 Math everywhere: %, angles & spirals

- **Percentages** — each card *level* adds **+10%**: `1 + 0.1 × (level - 1)` 📈
- **Probability** — the bot flips a coin to pick a lane: `rng() < 0.5` 🏛️
- **Angles & spirals** — a swarm spawns on a **golden-angle spiral** ($\approx 137.5^\circ$), the same pattern as seeds in a **sunflower!** 🌻

```
const a = i * 2.399963;           // golden angle (radians)
x = Math.cos(a) * r;  y = Math.sin(a) * r;  // even, pretty packing
```

Nature and games use the **same math**. 🤖

The #1 habit: test FIRST!

Red  → Green  → Refactor 

1.  Write a test for what you *want* — it fails (you haven't built it yet)
2.  Write the simplest code to make it pass
3.  Clean it up, keeping the test green

```
it("the firecracker hops backward after firing", () => {  
  const startY = firecracker.y;  
  run(battle, 1.5); // let time pass  
  expect(firecracker.y).toBeGreaterThan(startY + 1); // moved back!  
});
```

This game has **240+** of these checks running all the time! 

Run it on your computer

Three commands in the terminal:

```
npm install      # download the toolbox (once)
npm run dev      # play → open http://localhost:3101
npm test         # run all the checks
```

Play a friend on the same Wi-Fi:

```
npm run play     # prints a link to share
```

Code with an AI helper 🤖 (1/2)

Describe **exactly** what you want, like directing a robot. Try these:

Add a card

"Add a card called **Lava Hound** that flies, has 3000 health, attacks only buildings, moves slowly, and bursts into 6 lava pups when it dies. Write a test first."

Change the look

"Change the arena floor from sandy stone to green grass with flowers."

Code with an AI helper 🤖 (2/2)

✨ Add an ability

"Make the Wizard's fireball leave a ring of fire for 3 seconds that hurts enemies who walk through it."

🔊 Add a sound

"Play a cheerful trumpet when I destroy an enemy tower."

💡 **Always** ask the AI to **write a test first** and **keep Brain & Body separate**. That's how the pros do it!

Future ideas — YOUR turn!

Easier

- Invent 3 new cards · New arena theme (lava, ice, space) · Victory music · Rainbow troop trails

Medium

- A tornado spell · "Double trouble" mode · A smarter bot · A card-collection screen

Harder

- Battle emotes · A high-score leaderboard · A phone app · 3-player / team battles

Pick the one that excites you most!

Why this is awesome to learn

-  **Problem-solving** — big dreams into small steps
-  **Logic** — "if this, then that"
-  **Math** — distance, speed, time, angles (school stuff!)
-  **Creativity** — design characters and worlds
-  **Teamwork** — share code, like real studios

Minecraft, Roblox, Clash Royale — all started with someone's **first line of code**.

You can do this! 💪

"Everybody should learn to program a computer, because it teaches you how to think." — *Steve Jobs*

Run it → change one number → add a card → dream up a feature.

Have fun and stay curious. 🚀

Now go build something amazing!